

AI-Powered Revenue & Voice Copilot

System Architecture, Brain Design & Complete Data Flow

Petpooja Track • Hackathon 2025

1. System Overview

The system is composed of two tightly integrated modules sharing a common Brain layer. The Brain is responsible for all intelligence: generating response dialogue, detecting intent, maintaining conversation context, triggering upsells, building orders incrementally, and converting natural language to SQL queries against the PoS database.

Core Principle

Speech In → Brain (NLP + SQL + Memory + Upsell Logic) → Response Text + Order JSON → TTS Out

The Brain never touches audio directly. It receives text, emits text. All audio handling is at the edges.

SQLCoder-7b (via Cloudflare Workers AI / HuggingFace Inference) is the query layer — no manual SQL writing.

Component	Responsibility	Technology
STT Layer	Raw audio → text transcript	Browser Web Speech API / Whisper
Brain (Core)	Intent, dialogue, memory, upsell, order building, NL→SQL	Python + LLM + SQLCoder-7b API
Revenue Engine	Menu profitability, margin scoring, combo recommendations	Python + Pandas / ML
PoS Database	Menu items, prices, food cost, sales history	MySQL / PostgreSQL
TTS Layer	Response text → spoken audio to customer	Browser TTS / ElevenLabs / Gemini TTS
Order Manager	Accumulate cart, confirm, push JSON to PoS	Python + PoS REST API

2. The Brain — Detailed Design

The Brain is the central orchestration layer. Every incoming customer utterance passes through it and every outgoing response is generated by it. Below are its six sub-components and how they interact.

2.1 Sub-Components of the Brain

Sub-Component	Role	Key Inputs / Outputs
Intent Detector	Classify what the customer wants (order, query, modify, confirm, cancel, smalltalk)	Input: transcript text Output: intent label + confidence
Context Memory	Maintain full conversation state across turns (cart, preferences, history)	Input: all previous turns Output: enriched context object
Dialogue Generator	Generate natural, contextual response text in correct language	Input: intent + context + menu data Output: response string
Upsell Trigger Engine	Decide when and what to upsell based on Revenue Engine signals	Input: current cart + margin scores Output: upsell suggestion or null
Order Builder	Incrementally accumulate items, modifiers, quantities into a cart	Input: parsed item + modifier Output: updated cart JSON
NL→SQL Engine	Convert natural language menu queries to SQL via SQLCoder-7b	Input: question text + DB schema Output: SQL query → results

2.2 Context Memory Object Schema

This is the in-memory state object that persists across every turn of the conversation:

Field	Description / Example
session_id	Unique call/session identifier
language	Detected language: "en", "hi", "hinglish"
customer_name	Extracted if mentioned: "Rahul"
turn_history	List of {role, text, intent, timestamp} — full dialogue log
cart	List of {item_id, name, qty, modifiers, price, margin_score}
cart_total	Running total value of current order

Field	Description / Example
last_intent	Intent from previous turn for continuity logic
clarification_pending	Item/modifier that needs clarification: null or {field, options}
upsell_shown	List of item_ids already suggested (avoid repeating)
order_confirmed	Boolean — true after customer confirms summary
pos_order_id	Assigned after PoS push

3. Complete Data Flow — Turn by Turn

Every customer utterance triggers this exact sequence. Each step is numbered and corresponds to a module boundary.

#	Stage	What Happens	Input	Output	Tech / Component
0 1	Voice Capture	Customer speaks on the phone call. Raw audio stream is captured.	Phone audio stream	Raw audio bytes / stream	Browser STT / Twilio / WebRTC
0 2	Speech-to-Text	Audio converted to raw text transcript. Language detected (en/hi/hinglish).	Audio bytes	transcript: string lang: string	Web Speech API Whisper / Google STT
0 3	Brain: Intent Detection	Classify the utterance into an intent category. Extract item name, modifier, qty if present.	transcript context.turn_history	intent: string entities: {item, qty, modifier}	LLM prompt / fine-tuned classifier Regex fallback
0 4	Brain: Context Update	Merge new intent + entities into context memory. Resolve ambiguities using turn history.	intent entities context (prev)	context (updated) clarification_needed: bool	Python dict / Redis Session store
0 5	Brain: NL→SQL Query	If customer asks about menu/price/availability, convert question to SQL. Execute against PoS DB. Return structured result.	transcript / question DB schema string	SQL query string query_result : rows[]	SQLCoder-7b via Cloudflare / HuggingFace API DB connector
0 6	Brain: Order Builder	If intent is ORDER or MODIFY, update the cart. Add/remove items with modifiers and prices.	entities context.cart menu DB lookup	context.cart (updated) item_details: {price, margin}	Python cart logic PoS DB query
0 7	Revenue Engine Lookup	Fetch margin score and upsell candidates for items in cart. Run combo association check.	context.cart item_ids sales_history table	margin_scores[] upsell_candidates[] combo_suggestions[]	Revenue Engine module Precomputed scores table
0 8	Brain: Upsell Trigger	Decide if/what upsell to inject into response. Check upsell_shown to avoid repetition.	upsell_candidates[] context.upsell_shown	upsell_line: string or null	Python upsell logic Threshold rules

#	Stage	What Happens	Input	Output	Tech / Component
		Apply upsell_priority from Revenue Engine.	margin_scores		
09	Brain: Dialogue Generator	Compose full response text: answer + upsell + next prompt. Ensure multi-language output matches detected lang.	intent query_result or cart upsell_line context lang	response_text: string	LLM (Gemini/GPT via API) Template fallback
10	Text-to-Speech	Convert response text to audio. Play audio back to customer on call.	response_text lang	audio bytes / stream	Gemini TTS / ElevenLabs Browser TTS (dev)
11	Order Confirmation	When intent = CONFIRM, read full cart summary. Wait for customer yes/no.	context.cart cart_total	confirmation_prompt text awaiting_confirmation flag	Dialogue Generator State machine
12	PoS Push & KOT	Serialize cart to structured JSON order. POST to PoS REST API. Auto-generate KOT.	context.cart session_id customer_name	order_json pos_order_id KOT created	PoS REST API JSON serializer

4. Intent Types & Brain Routing

The Intent Detector outputs one of these intents. The Brain routes to different sub-modules depending on the intent.

Intent	Example Utterance	Brain Routes To	Response Action
ORDER_ADD	"Give me 2 paneer butter masala"	Order Builder → Upsell Trigger → Dialogue	Confirm item + upsell
ORDER_MODIFY	"Remove the extra spice"	Order Builder (modify) → Dialogue	Confirm change
ORDER_REMOVE	"Cancel the naan"	Order Builder (remove) → Dialogue	Confirm removal
QUERY_MENU	"What do you have for starters?"	NL→SQL → Dialogue	Read out options
QUERY_PRICE	"How much is the biryani?"	NL→SQL → Dialogue	State price
QUERY_AVAILABILITY	"Is the fish curry available today?"	NL→SQL → Dialogue	Yes/No + suggest alt
CLARIFY_RESPONSE	"Medium spice" (after clarification prompt)	Context Update → Order Builder	Complete pending item
CONFIRM_ORDER	"Yes, confirm my order"	Order Confirmation → PoS Push	Summary + push JSON
CANCEL_ORDER	"Actually cancel everything"	Order Builder (clear) → Dialogue	Clear cart + confirm
SMALLTALK	"How are you?" / "Thanks"	Dialogue Generator only	Polite response

5. NL→SQL Engine — SQLCoder-7b Integration

The NL→SQL engine uses defog/sqlcoder-7b-2 to convert customer questions and internal lookups into SQL queries against your PoS database. This replaces all manual query writing.

5.1 How It Works

- Customer asks: "What's the cheapest vegetarian main course?"
- Brain extracts question + constructs prompt with DB schema
- POST to SQLCoder API (Cloudflare Workers AI or HuggingFace Inference)
- SQL returned → executed against PoS DB → results fed to Dialogue Generator

5.2 SQLCoder Prompt Template

Prompt Structure Sent to SQLCoder-7b

Task

Generate a SQL query to answer the following question.

Database Schema

```
CREATE TABLE menu_items (item_id INT, name VARCHAR, category VARCHAR, selling_price DECIMAL, food_cost DECIMAL, is_available BOOLEAN, is_veg BOOLEAN);
CREATE TABLE order_lines (id INT, order_id INT, item_id INT, qty INT, created_at DATETIME);
CREATE TABLE combos (combo_id INT, item_a INT, item_b INT, frequency INT, margin_lift DECIMAL);
```

Question

{customer_question_or_internal_lookup}

SQL

(model completes this)

5.3 API Calls — Two Options

Option	Endpoint	Notes
Cloudflare Workers AI	POST /client/v4/accounts/{id}/ai/run/@cf/defog/sqlcoder-7b-2	Free tier available. Fastest. Recommended.
HuggingFace Inference	POST api-inference.huggingface.co/models/defog/sqlcoder-7b-2	Free tier with cold starts. PRO \$9/mo for warm model.

5.4 When NL→SQL Is Triggered

- Customer queries menu (price, availability, category, recommendation)
- Revenue Engine needs margin/sales data for upsell scoring (internal query)
- Order Builder needs item_id and food_cost for a named item (internal lookup)
- Combo Engine checks association data for current cart items (internal query)

6. Revenue Intelligence Engine

Runs as a background service. Pre-computes scores that the Brain uses during calls for upsell decisions. Also powers the dashboard.

6.1 Calculations

Metric	Formula / Logic
Contribution Margin	$\text{selling_price} - \text{food_cost}$ (per item)
Contribution Margin %	$(\text{selling_price} - \text{food_cost}) / \text{selling_price} \times 100$
Popularity Score	$\text{units_sold_last_30d} / \text{max_units_sold} \times 100$ (normalized 0–100)
BCG Quadrant Label	High Margin + High Pop = Star High Margin + Low Pop = Hidden Star Low Margin + High Pop = Risk Low + Low = Dog
Upsell Priority Score	$(\text{margin_pct} \times 0.6) + (1 - \text{popularity_score}/100 \times 0.4)$ — promotes high margin under-promoted items
Combo Lift	Apriori / FP-Growth association rules on order_lines — items frequently bought together
Price Elasticity	Regression of price changes vs volume (optional advanced)

6.2 Output: Precomputed Scores Table

The Revenue Engine writes results to a scores table in your DB, which the Brain reads in real time during calls:

Column	Description
item_id	FK to menu_items
margin_score	Contribution margin in ₹
margin_pct	CM as percentage of selling price
popularity_score	Normalized 0–100
quadrant	Star / Hidden Star / Risk / Dog
upsell_priority	Float 0–1, higher = upsell first
top_combos	JSON array of item_ids frequently paired with this item
last_computed	Timestamp of last score run

7. Upsell Trigger Logic — Decision Flow

The Upsell Trigger runs after Order Builder on every ORDER_ADD intent. It decides whether to upsell and what to suggest.

Step	Logic
Step 1: Timing Gate	Only trigger upsell if turn_count >= 2 (not on first utterance) AND cart has at least 1 item
Step 2: Avoid Repetition	Filter out any item_id already in context.upsell_shown
Step 3: Combo Check	Look up top_combos for latest added item. If a combo item is not in cart → candidate
Step 4: Hidden Star Check	From scores table: get all Hidden Star items not in cart, sorted by upsell_priority DESC
Step 5: Select Top 1	Pick the highest priority candidate from Step 3 or Step 4
Step 6: Inject into Dialogue	Pass upsell_item to Dialogue Generator as upsell_line. Mark item in upsell_shown.
Step 7: No Double Upsell	Never upsell two items in the same turn. One suggestion per response maximum.

Example Upsell Dialogue Injection

Customer: "I'll have a chicken biryani please."

Order Builder: adds chicken biryani (item_id: 42) to cart

Revenue Engine lookup: item 42's top_combo = [55 (raita), 61 (gulab jamun)]

Upsell Trigger: raita (55) is Hidden Star, upsell_priority: 0.87, not yet shown

Dialogue Generator output: "Great choice! Your chicken biryani has been added.

Many customers love it with our house-made raita — shall I add one for just ₹49?"

8. Order JSON Structure — PoS Push Payload

After the customer confirms, the Brain serializes the cart into this JSON and POSTs it to the PoS REST API.

Order JSON Schema

```
{
  "order_id": "ORD-20250601-0042",
  "session_id": "CALL-9876543210-1234",
  "channel": "voice",
  "language": "hinglish",
  "customer_name": "Rahul",
  "customer_phone": "+91-9876543210",
  "items": [
    {
      "item_id": 42,
      "name": "Chicken Biryani",
      "qty": 1,
      "modifiers": [{"type": "spice", "value": "medium"}],
      "unit_price": 280,
      "line_total": 280
    },
    {
      "item_id": 55,
      "name": "House Raita",
      "qty": 1,
      "modifiers": [],
      "unit_price": 49,
      "line_total": 49
    }
  ],
  "order_total": 329,
  "status": "confirmed",
  "upsell_accepted": ["item_id:55"],
  "created_at": "2025-06-01T13:42:00Z"
}
```

9. Suggested Database Schema

This is the minimal schema the Brain's NL→SQL engine and Revenue Engine operate against. Pass this as the schema string to SQLCoder.

Table	Key Columns
menu_items	item_id, name, category, selling_price, food_cost, is_available, is_veg, is_vegan, image_url
modifiers	modifier_id, item_id, modifier_type (spice/size/addon), options JSON, price_delta
orders	order_id, channel, customer_phone, order_total, status, created_at
order_lines	id, order_id, item_id, qty, modifiers JSON, unit_price, line_total
revenue_scores	item_id, margin_score, margin_pct, popularity_score, quadrant, upsell_priority, top_combos JSON, last_computed
combos	combo_id, item_a, item_b, co_occurrence_count, confidence, lift, last_updated
sessions	session_id, customer_phone, language, cart JSON, turn_history JSON, status, created_at

10. Recommended Tech Stack

Layer	Technology	Why
STT	Web Speech API (browser) / OpenAI Whisper	Browser = free, no setup. Whisper = more accurate, offline possible.
TTS	Gemini TTS / ElevenLabs / Browser TTS	Adapter pattern — swap freely. Start with Browser TTS for dev.
Brain Dialogue LLM	Gemini 1.5 Flash / GPT-4o-mini	Fast, cheap, multilingual. Flash = best price/quality for dialogue.
NL→SQL	SQLCoder-7b via Cloudflare Workers AI	Free tier, no local GPU, purpose-built for SQL generation.
Revenue Engine	Python + Pandas + Scikit-learn	Apriori for combos, margin calcs, simple ML classification.
Backend / Brain API	FastAPI (Python)	Async, fast, easy to containerize, auto docs with Swagger.
Database	PostgreSQL	Robust, JSON support, good for analytics queries.
Session / Context	Redis	In-memory, fast, perfect for per-

Layer	Technology	Why
Store		session context objects.
Frontend / Dashboard	React + Recharts	Revenue dashboard, order monitor, demo interface.
Call Handling	Twilio Voice / WebRTC	For real phone call integration. Browser demo uses WebRTC.

11. Judging Criteria Alignment

Weight	Criteria	What to Demo	Your System's Answer
25%	Problem Solving	Revenue analysis + menu classification	Revenue Engine: margin scores, BCG quadrant, Hidden Stars table
25%	AI / Tech Innovation	NLP quality, upsell logic, voice pipeline	SQLCoder-7b NL→SQL + LLM dialogue + upsell trigger system
20%	UX & Demo	Live voice order on call	React dashboard + live voice demo with TTS/STT in browser
20%	Real-World Impact	AOV uplift potential	Show upsell acceptance rate, average order value before/after
10%	Bonus Features	Hinglish + KOT auto creation	Multi-language dialogue + PoS JSON push + auto KOT

Key Differentiator to Highlight in Presentation

Your NL→SQL layer (SQLCoder-7b) means the system can answer ANY menu or analytics query without

pre-programmed responses. A judge asking "what if a customer asks something unexpected?" is fully covered.

Combined with the adapter-pattern TTS/STT, you can swap any provider without touching business logic.

This shows real engineering maturity beyond a typical hackathon prototype.